

PSE-NCTCS-CONFORMANCE-01

Null-Centered Toroidal Control Closure Layer

Formale Implementierungsspezifikation v0.1

Sebastian Klemm

Technische Ausarbeitung fuer Coding-Agent-Implementierung

Mai 2026

Zusammenfassung

Diese Spezifikation definiert einen NCTCS-Konformitaets- und Kontrollabschluss-Layer fuer den vorhandenen PSE-Stack. Der Layer erzeugt keine neue Engine, keine neue Zellwelt, keinen neuen Tensor und keinen alternativen Commit-Pfad. Er liest vorhandene Artefakte aus PSE-Core, pse-traverse, Cognition, PhaseMatrix, Dual-Fabric Stitch, TPT-MTL und Validation-Runner, prueft sie gegen den Null-Centered Toroidal Control Substrate, klassifiziert die erreichte Konformitaetsklasse C0 bis C4 und verdichtet den gueltigen Systemzustand in einen content-addressed **MacroControlState**. Ziel ist ein einheitlicher Kontrollabschluss: lokale Resonanz, ephemere Fabrics, Gate-Entscheidungen, Tensorrevisionen, Trace, Replay, Evaluation und reale Domaenenvalidierung muenden in einen einzigen auditierbaren Makrozustand.

Inhaltsverzeichnis

1	Status, Zweck und Nicht-Ziele	4
1.1	Status	4
1.2	Zweck	4
1.3	Nicht-Ziele	4
2	Architekturposition	4
2.1	Schichtposition	4
2.2	Integration als Closure, nicht als Parallelarchitektur	5
3	Formal Mapping auf vorhandene PSE-Artefakte	5
4	Konformitätsklassen	6
4.1	C0 bis C4	6
4.2	Validation-Status vs. NCTCS-Klasse	6
5	Datenmodelle	7
5.1	Run Descriptor	7
5.2	NullCenter und Phase Projection	7
5.3	Visibility und Candidate Audit	8
5.4	Materialization Audit	8
5.5	Trace Replay Contract	9
5.6	MacroControlState	9
5.7	Conformance Report und Closure Bundle	9
6	Gate Outcome Taxonomy	10
6.1	Outcome-Klassen	10
6.2	Normative Regeln	10
7	Pipeline	11
7.1	Referenzfluss	11
7.2	Pseudocode	11
8	Integration in Validation Runner	12
8.1	CLI-Erweiterungen	12
8.2	One-Button-Regel	12
9	Evaluation-Matrix-Integration	12
9.1	Neue Metriken	12
9.2	Schlussformel-Erweiterung	13

10 Tests	13
10.1 Unit Tests	13
10.2 Integration Tests	13
10.3 Negative Tests	14
11 Datei- und Modulstruktur	14
12 Definition of Done	14
13 Coding-Agent-Auftrag	14
14 Schlussformel	15

1 Status, Zweck und Nicht-Ziele

1.1 Status

Dieses Dokument ist eine normative Implementierungsspezifikation. Die Begriffe **MUST**, **MUST NOT**, **SHOULD** und **MAY** sind verbindlich fuer Konformitaet.

1.2 Zweck

PSE-NCTCS-CONFORMANCE-01 schliesst den vorhandenen PSE-Stack formal ab. Der Layer prueft, ob die vorhandenen Laufzeit- und Validierungsartefakte den NCTCS-Kontrollpfad erfuellen:

$$K_0 \xrightarrow{\pi_0} \theta_0 \xrightarrow{\Phi_\omega^t} \theta_t \xrightarrow{R_{\epsilon}} Vis_t \xrightarrow{Cand} Q_t \xrightarrow{H} F_H(t) \xrightarrow{D} SC_t \xrightarrow{G} CU_t \xrightarrow{U} F_T(t+1) \xrightarrow{\tau, M} M(t+1).$$

Kerninvariante

Resonanz darf im ephemeren Fabric frei auftreten. Persistente Struktur darf nur durch gate-bound, evidence-linked, trace-preserving und replay-identische Tensorupdates materialisieren. Der Makro-Kontrollzustand entsteht ausschliesslich aus NullCenter-Referenz, persistenter Tensorhistorie, Trace, Policies, Certificates und Evaluationsergebnis.

1.3 Nicht-Ziele

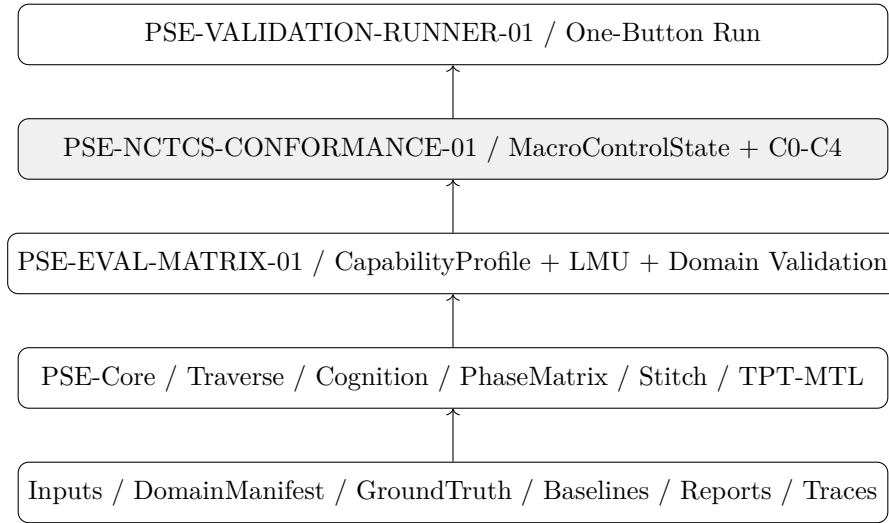
Diese Spezifikation definiert nicht:

- einen neuen PSE-Core;
- eine neue PhaseMatrix-Implementierung;
- einen neuen FieldTensor-Stitcher;
- eine neue TPT-MTL-Triangulationsschicht;
- einen neuen Eval-Matrix-Kern;
- einen neuen Commit-Pfad, **SemanticCrystal**-Erzeugung oder **FinalizedEmission**;
- ein Netzwerk-, Kommunikations-, Aktorik- oder Produkt-API-Protokoll.

2 Architekturposition

2.1 Schichtposition

NCTCS-Conformance liegt ueber den vorhandenen Reports, aber unterhalb jeder externen Produkt- oder UI-Schicht. Es ist ein Audit- und Closure-Layer.



2.2 Integration als Closure, nicht als Parallelarchitektur

Der Layer **MUST** vorhandene Artefakte konsumieren und pruefen. Er **MUST NOT** vorhandene Layer duplizieren. Insbesondere:

- `CellSubstrateCycleReport` bleibt Quelle fuer lokale Resonanz-/Clusterereignisse.
- `StitchCycleBundle` bleibt Quelle fuer Fabric-H/Fabric-T/Tensorupdate-Logik.
- `TptMtlBundle` bleibt Quelle fuer topologische Orientierung und Mesh-/Lift-Reports.
- `EvaluationSummaryReport` bleibt Quelle fuer empirische Verbesserung, LMU, Replay und Safety.
- `ValidationRunBundle` bleibt Quelle fuer Build/Test/Benchmark/Domain-Status.

3 Formal Mapping auf vorhandene PSE-Artefakte

NCTCS	PSE-Quelle	Bedeutung im Closure-Layer
B_0	RunDescriptor / RepoState / NullCenter declaration	Pre-dynamic reference layer; kein dynamischer Zustand.
K_0	NullCenterRef / Validation-Run root / matrix id	Exogener Anker; darf keine lokale Zelle und kein Agent sein.
π_0	NullProjectionAudit	Projektion von K_0 in T^n ; strikt getrennt von K_0 .
T^n	Horizon / TPT / phase descriptor	Hypertoroidaler Phasenraum fuer Sichtbarkeit.
Φ_ω	PhaseScheduler / HorizonRay / TPT window	Globaler Phasenfluss fuer Timing, nicht Wahrheit.
P	PhaseCell / CellPool / Phase-Subnet	Lokales Resonanzzellfeld.
R_ϵ	PhaseVisibilityAudit	Sichtbarkeitsoperator fuer Zellen/Cluster/Kandidaten.
F_H	ResonanceFabricState / Cell-SubstrateCycleReport	Ephemeres Resonanzfabric.
F_T	FieldTensorState / StitchCycleBundle	Persistenter Feldtensor.

NCTCS	PSE-Quelle	Bedeutung im Closure-Layer
G	StitcherGate / GateReports / Eval gates	Fail-closed Gate-Kaskade.
τ	Trace / Evidence / ReplayManifest / Ledger	Trace-/Evidence-/Replay-Vertrag.
M	MacroControlState	Verdichteter Makro-Kontrollzustand.

4 Konformitaetsklassen

4.1 C0 bis C4

Klasse	Name	Bedingung
C0	Formal Typed NCTCS	Tuple-Struktur existiert, $K_0 \in B_0$ und $K_0 \notin T^n$ sind nachweisbar.
C1	Phase-Gated Visibility	C0 + Kandidatenbildung impliziert phase-gated visibility.
C2	Gate-Bound Materialization	C1 + ephemeres Fabric kann persistenten Tensor nicht direkt mutieren; Gate-Fail erhaelt Tensorzustand.
C3	Auditable Tensor Substrate	C2 + jede Persistenz impliziert TraceReady, EvidenceReady und ReplayReady.
C4	Macro-Control Substrate	C3 + gueltiger MacroControlState aus K_0 , F_T , Trace, Policies, Certificates und ValidationResult.

4.2 Validation-Status vs. NCTCS-Klasse

NCTCS-Klasse und Validierungsstatus sind getrennt. Ein Run kann C4 erreichen und trotzdem nur STRUCTURAL_PASS sein, wenn keine reale Domaenenvalidierung ausgefuehrt wurde.

Status	Bedeutung
INVALID	Build, Tests, Replay, Ledger, Reports oder NCTCS-Invarianten defekt.
INTERNAL_PASS	Interne technische Pruefung bestanden, aber noch keine vollstaendige strukturelle Auswertung.
STRUCTURAL_PASS	Replay, Ablation, Eval-Matrix und NCTCS-Coverage intern geschlossen.
DOMAIN_READY	Reales DomainManifest mit Splits, GroundTruth und Baselines ist vorhanden und validiert.
DOMAIN_VALIDATED	Mindestens eine reale Domaene wurde ausgefuehrt und replaybar ausgewertet.
DIAGNOSTIC_FINDING	Gueltiger Lauf ohne belastbaren Vorteil oder mit Tradeoff.
EMPIRICAL_IMPROVEMENT	Reale Domaenenvalidierung erfuellt die Eval-Matrix-Schlussformel.

Reale Domaenvalidierung

Ein Run **MUST NOT** als DOMAIN_VALIDATED oder EMPIRICAL_IMPROVEMENT klassifiziert werden, wenn kein reales DomainValidationManifest mit Calibration/Validation/Test-Split, GroundTruthProfile und BaselineProfile ausgeführt wurde. Ohne reale Domaene ist der hoechste zulaessige Status STRUCTURAL_PASS.

5 Datenmodelle

5.1 Run Descriptor

```
pub struct NctcsRunDescriptor {
  pub run_id: StableId,
  pub closure_profile: String,          // "PSE-NCTCS-CONFORMANCE-01"
  pub canonicalization_version: String,
  pub source_validation_bundle_hash: Hash256,
  pub source_eval_summary_hash: Option<Hash256>,
  pub source_stitch_bundle_hash: Option<Hash256>,
  pub source_tpt_bundle_hash: Option<Hash256>,
  pub thresholds: NctcsThresholds,
  pub policies: NctcsPolicies,
  pub operator_versions: BTreeMap<String, String>,
  pub evidence_refs: Vec<EvidenceRef>,
}

pub struct NctcsThresholds {
  pub min_visibility_coverage: Fixed,
  pub min_gate_coverage: Fixed,
  pub min_trace_coverage: Fixed,
  pub min_replay_identity: Fixed,
  pub max_unmapped_artifact_rate: Fixed,
  pub max_coherence_without_truth_rate: Fixed,
}

pub struct NctcsPolicies {
  pub require_real_domain_for_complete_validation: bool,
  pub require_tensor_history_for_c4: bool,
  pub require_eval_matrix_for_macro_state: bool,
  pub require_sorted_artifacts_before_hash: bool,
  pub fail_on_unmapped_persistent_artifact: bool,
}
```

5.2 NullCenter und Phase Projection

```
pub struct NullCenterRef {
  pub null_center_id: Hash256,
  pub pre_dynamic_layer_id: Hash256,
  pub declared_exogenous: bool,
  pub not_phase_state: bool,
  pub not_agent: bool,
  pub evidence_refs: Vec<EvidenceRef>,
}

pub struct NullProjectionAudit {
  pub audit_id: Hash256,
```

```

    pub null_center_id: Hash256,
    pub projected_phase_hash: Hash256,
    pub projection_distinction_passed: bool,
    pub phase_manifold_dim: u32,
    pub phase_manifold_hash: Hash256,
}

pub struct ToroidalPhaseFlowAudit {
    pub audit_id: Hash256,
    pub phase_flow_hash: Hash256,
    pub tick: u64,
    pub theta_t_hash: Hash256,
    pub visibility_timing_only: bool,
}

```

5.3 Visibility und Candidate Audit

```

pub struct PhaseVisibilityAudit {
    pub audit_id: Hash256,
    pub theta_t_hash: Hash256,
    pub visible_cell_refs: Vec<Hash256>,
    pub invisible_cell_refs: Vec<Hash256>,
    pub visibility_coverage: Fixed,
    pub passed: bool,
}

pub struct CandidateFormationAudit {
    pub audit_id: Hash256,
    pub candidate_refs: Vec<Hash256>,
    pub rejected_refs: Vec<Hash256>,
    pub candidate_requires_visibility_passed: bool,
    pub activation_gate_coverage: Fixed,
    pub boundary_ready_coverage: Fixed,
    pub evidence_ready_coverage: Fixed,
    pub trace_replay_ready_coverage: Fixed,
    pub passed: bool,
}

```

5.4 Materialization Audit

```

pub struct MaterializationAudit {
    pub audit_id: Hash256,
    pub ephemeral_fabric_hash: Hash256,
    pub tensor_before_hash: Hash256,
    pub stitch_candidate_hashes: Vec<Hash256>,
    pub accepted_update_hashes: Vec<Hash256>,
    pub rejected_candidate_hashes: Vec<Hash256>,
    pub gate_report_hashes: Vec<Hash256>,
    pub tensor_after_hash: Hash256,
    pub tensor_unchanged_on_no_update: bool,
    pub no_direct_fabric_to_tensor_mutation: bool,
    pub passed: bool,
}

```

5.5 Trace Replay Contract

```
pub struct TraceReplayContractReport {
  pub report_id: Hash256,
  pub trace_head: Hash256,
  pub evidence_root: Hash256,
  pub gate_history_root: Hash256,
  pub replay_manifest_hash: Hash256,
  pub canonicalization_hash: Hash256,
  pub replay_identity: Fixed,
  pub replay_ready_required_for_gate_pass: bool,
  pub passed: bool,
}
```

5.6 MacroControlState

```
pub struct MacroControlState {
  pub macro_state_id: Hash256,
  pub run_id: StableId,
  pub null_center_ref: Hash256,
  pub field_tensor_ref: Option<Hash256>,
  pub trace_head: Hash256,
  pub orientation_map_hash: Option<Hash256>,
  pub active_horizons: Vec<Hash256>,
  pub blocked_horizons: Vec<Hash256>,
  pub attractor_map_hash: Option<Hash256>,
  pub aggregate_gate_status: AggregateGateStatus,
  pub conformance_class: NctcsConformanceClass,
  pub validation_status: ValidationClosureStatus,
  pub eval_summary_hash: Option<Hash256>,
  pub domain_validation_hash: Option<Hash256>,
  pub evidence_refs: Vec<EvidenceRef>,
}

pub struct AggregateGateStatus {
  pub total_gates: u64,
  pub passed_gates: u64,
  pub failed_gates: u64,
  pub held_gates: u64,
  pub quarantined_candidates: u64,
  pub no_update_count: u64,
  pub gate_coverage: Fixed,
}
```

5.7 Conformance Report und Closure Bundle

```
pub enum NctcsConformanceClass {
  C0_FormalTyped,
  C1_PhaseGatedVisibility,
  C2_GateBoundMaterialization,
  C3_AuditTableTensor,
  C4_MacroControl,
}

pub enum ValidationClosureStatus {
```

```

    Invalid,
    InternalPass,
    StructuralPass,
    DomainReady,
    DomainValidated,
    DiagnosticFinding,
    EmpiricalImprovement,
}

pub struct NctcsConformanceReport {
    pub report_id: Hash256,
    pub rd_hash: Hash256,
    pub c0: ConformanceCheck,
    pub c1: ConformanceCheck,
    pub c2: ConformanceCheck,
    pub c3: ConformanceCheck,
    pub c4: ConformanceCheck,
    pub reached_class: NctcsConformanceClass,
    pub validation_status: ValidationClosureStatus,
    pub unresolved_obligations: Vec<ProofObligationRef>,
}

pub struct NctcsClosureBundle {
    pub bundle_id: Hash256,
    pub rd: NctcsRunDescriptor,
    pub null_center: NullCenterRef,
    pub projection: NullProjectionAudit,
    pub phase_flow: ToroidalPhaseFlowAudit,
    pub visibility: PhaseVisibilityAudit,
    pub candidate_audit: CandidateFormationAudit,
    pub materialization: MaterializationAudit,
    pub trace_replay: TraceReplayContractReport,
    pub macro_state: Option<MacroControlState>,
    pub conformance: NctcsConformanceReport,
}

```

6 Gate Outcome Taxonomy

6.1 Outcome-Klassen

```

pub enum NctcsGateOutcome {
    Pass,
    Hold,
    Refine,
    Reject,
    Quarantine,
    NoUpdate,
    HandoffReady,
}

```

6.2 Normative Regeln

- Nur Pass **MAY** persistente Tensorrevision erzeugen.

- Hold, Refine, Reject, Quarantine, NoUpdate und HandoffReady **MUST** nicht-materialisierend sein.
- Reject und Hold sind typisierte Kontrollantworten, keine Systemfehler.
- HandoffReady **MAY** externe Finalisierung vorbereiten, **MUST NOT** aber selbst committen.

7 Pipeline

7.1 Referenzfluss

1. Lade ValidationRunBundle, EvaluationSummaryReport, optional StitchCycleBundle, optional TptMtlBundle.
2. Erzeuge oder lese NctcsRunDescriptor.
3. Resolviere NullCenterRef; pruefe Exogenitaet.
4. Erzeuge NullProjectionAudit; pruefe $K_0 \neq \pi_0(K_0)$.
5. Erzeuge ToroidalPhaseFlowAudit; pruefe, dass PhaseFlow Timing und nicht Wahrheit erzeugt.
6. Erzeuge PhaseVisibilityAudit; pruefe Sichtbarkeitsfenster.
7. Erzeuge CandidateFormationAudit; pruefe, dass Kandidaten Sichtbarkeit, Aktivierung, Boundary, Evidence und Trace voraussetzen.
8. Erzeuge MaterializationAudit; pruefe Fabric-Isolation, Stitch-Gates, Tensor-Delta und NoUpdate-Stabilitaet.
9. Erzeuge TraceReplayContractReport; pruefe ReplayIdentity und Gate-Replay-Kopplung.
10. Erzeuge MacroControlState, sofern C4 moeglich ist.
11. Erzeuge NctcsConformanceReport; klassifiziere C0-C4.
12. Erzeuge NctcsClosureBundle; pruefe byte-identische Replay-Rekonstruktion.

7.2 Pseudocode

```
pub fn close_nctcs(run: ValidationRunBundle, rd: NctcsRunDescriptor)
  -> NctcsClosureBundle
{
  let k0 = resolve_null_center(&run, &rd);
  let projection = audit_null_projection(&k0, &rd);
  let phase = audit_phase_flow(&projection, &run, &rd);
  let visibility = audit_phase_visibility(&phase, &run, &rd);
  let candidates = audit_candidate_formation(&visibility, &run, &rd);
  let materialization = audit_materialization_path(&candidates, &run, &rd);
  let trace_replay = audit_trace_replay_contract(&materialization, &run, &rd);

  let conformance = classify_conformance(
    &k0,
    &projection,
    &visibility,
    &candidates,
    &materialization,
    &trace_replay,
    &run,
    &rd,
  );

  let macro_state = if conformance.reached_class == NctcsConformanceClass::
  C4_MacroControl {
    Some(build_macro_control_state(&k0, &materialization, &trace_replay, &run, &rd))
  }
```

```

    } else {
        None
    };

    build_closure_bundle(rd, k0, projection, phase, visibility,
                        candidates, materialization, trace_replay,
                        macro_state, conformance)
}

```

8 Integration in Validation Runner

8.1 CLI-Erweiterungen

Der bestehende Validation Runner **MUST** NCTCS als Closure-Phase integrieren.

```

pse-validation-runner nctcs-close \
  --run validation_runs/<commit>/validation_bundle.json \
  --out validation_runs/<commit>/nctcs_closure_bundle.json

pse-validation-runner nctcs-replay \
  --bundle validation_runs/<commit>/nctcs_closure_bundle.json

pse-validation-runner nctcs-verify \
  --bundle validation_runs/<commit>/nctcs_closure_bundle.json

pse-validation-runner run-all \
  --domain domain_manifest.json \
  --with-nctcs \
  --out validation_runs/<commit>/

```

8.2 One-Button-Regel

`run-all -with-nctcs` **MUST** nach Build, Tests, Benchmarks, Eval-Matrix und realer Domaenen-validierung automatisch `nctcs-close`, `nctcs-replay` und `nctcs-verify` ausfuehren.

9 Evaluation-Matrix-Integration

9.1 Neue Metriken

Die Eval-Matrix **SHOULD** folgende NCTCS-Metriken registrieren:

Metrik	Bedeutung
<code>nctcs_conformance_class_score</code>	Numerische Abbildung C0=0.25, C1=0.40, C2=0.60, C3=0.80, C4=1.00.
<code>nctcs_visibility_candidate_completeness</code>	Anzahl Kandidaten, deren Sichtbarkeitsbedingung nachweislich erfuehlt war.
<code>nctcs_no_direct_persistence_rate</code>	Anteil ephemerer Ereignisse ohne direkte Tensor-Mutation.
<code>nctcs_gate_bound_revision_rate</code>	Anteil Tensorrevisionen mit vollstaendig bestandenem Gatepfad.

Metrik	Bedeutung
<code>nctcs_trace_replay_contract_rate</code>	Anteil persistenter Artefakte mit Trace, Evidence, GateHistory und ReplayManifest.
<code>nctcs_macro_state_validity</code>	1, wenn MacroControlState aus Tensorhistorie statt aus Resonanz/Fabric direkt abgeleitet ist.
<code>nctcs_coherence_truth_separation_rate</code>	Anteil Reports, die Kohärenz nicht als Wahrheit klassifizieren.
<code>nctcs_domain_validation_required_completion</code>	Empirical-Status nur bei realer Domainenvalidierung vergeben wurde.

9.2 Schlussformel-Erweiterung

Ein Run darf nur dann als `EMPIRICAL_IMPROVEMENT` gelten, wenn zusaetzlich zur vorhandenen Eval-Matrix-Schlussformel gilt:

$$NCTCSClass \geq C3 \wedge ReplayContract = 1 \wedge NoDirectPersistence = 1.$$

Wenn der Run einen `MacroControlState` als Produkt des Kontrollabschlusses behauptet, gilt zusaetzlich:

$$NCTCSClass = C4.$$

10 Tests

10.1 Unit Tests

1. `null_center_is_not_phase_state`
2. `projection_distinguishes_k0_from_theta0`
3. `candidate_requires_phase_visibility`
4. `visibility_does_not_imply_persistence`
5. `ephemeral_fabric_cannot_directly_mutate_tensor`
6. `gate_failure_preserves_tensor_state`
7. `replay_ready_required_for_gate_pass`
8. `macro_state_requires_tensor_history_and_trace`
9. `coherence_is_not_truth`
10. `hold_and_reject_are_non_materializing`

10.2 Integration Tests

1. `nctcs_closure_reaches_c2_without_tensor_trace`
2. `nctcs_closure_reaches_c3_with_trace_replay_contract`
3. `nctcs_closure_reaches_c4_with_macro_control_state`
4. `nctcs_replay_byte_identical`
5. `nctcs_run_all_with_domain_emits_closure_bundle`
6. `nctcs_without_domain_cannot_claim_empirical_improvement`
7. `nctcs_metrics_registered_in_eval_matrix`
8. `macro_state_changes_when_tensor_history_changes`

10.3 Negative Tests

1. Direkte Fabric-zu-Tensor-Mutation muss fehlschlagen.
2. Kandidat ohne Sichtbarkeit darf C1 nicht bestehen.
3. Tensorrevision ohne Gate-Pass darf C2 nicht bestehen.
4. Persistenz ohne Trace/Evidence/Replay darf C3 nicht bestehen.
5. MacroControlState direkt aus Resonanz oder ephemerem Fabric darf C4 nicht bestehen.
6. Run ohne reale Domaene darf nicht DOMAIN_VALIDATED oder EMPIRICAL_IMPROVEMENT werden.

11 Datei- und Modulstruktur

```
crates/pse-validation-runner/src/nctcs/
  mod.rs
  primitives.rs
  run_descriptor.rs
  null_center.rs
  phase_visibility.rs
  candidate_audit.rs
  materialization_audit.rs
  trace_replay_contract.rs
  macro_control_state.rs
  conformance.rs
  metrics.rs
  report.rs
  replay.rs
  pipeline.rs
```

Falls `pse-validation-runner` noch nicht als Crate existiert, **MAY** die NCTCS-Module im neuen Runner-Crate angelegt werden. Falls der Runner bereits existiert, **MUST** NCTCS als Submodul integriert werden, nicht als separates Paralleltool.

12 Definition of Done

Eine Implementierung ist abgeschlossen, wenn:

1. `cargo fmt -all - -check` sauber ist.
2. `cargo clippy -workspace -all-targets -locked` sauber ist.
3. `cargo test -workspace -locked` besteht.
4. `cargo doc -workspace -no-deps -locked` warning-free ist.
5. `nctcs-close`, `nctcs-replay` und `nctcs-verify` funktionieren.
6. `run-all -with-nctcs -domain <manifest>` einen ClosureBundle erzeugt.
7. C0-C4-Konformitaetsklassifikation getestet ist.
8. Ohne reale Domaene kein EMPIRICAL_IMPROVEMENT vergeben werden kann.
9. NCTCS-Metriken in der Eval-Matrix sichtbar sind.
10. `final_report.md` den erreichten Validierungsstatus, die NCTCS-Klasse, offene Proof Obligations und den MacroControlState-Hash ausweist.

13 Coding-Agent-Auftrag

Implementiere PSE-NCTCS-CONFORMANCE-01 als Closure-Layer im Validation Runner.

PRIMARY SPEC:

- pse_nctcs_conformance_spec_v0_1.pdf

ZIEL:

Der vorhandene PSE-Stack soll nach einem ValidationRun in einen NCTCS-ClosureBundle verdichtet werden. Der Layer erzeugt keine neue Engine und keinen neuen Commit-Pfad. Er prueft vorhandene Artefakte, klassifiziert C0-C4 und baut bei C4 einen MacroControlState.

MUSS:

- NCTCS-Module im Validation-Runner integrieren.
- NctcsRunDescriptor, NullCenterRef, NullProjectionAudit, PhaseVisibilityAudit, CandidateFormationAudit, MaterializationAudit, TraceReplayContractReport, MacroControlState, NctcsConformanceReport und NctcsClosureBundle implementieren.
- CLI: nctcs-close, nctcs-replay, nctcs-verify integrieren.
- run-all --with-nctcs nach internen Checks, Eval-Matrix und DomainRun ausfuehren.
- C0-C4 klassifizieren.
- Ohne reales DomainValidationManifest maximal STRUCTURAL_PASS erlauben.
- NCTCS-Metriken in pse-eval-matrix registrieren oder adapterfaehig machen.
- Replay byte-identisch pruefen.

DARF NICHT:

- keine neue PhaseMatrix, kein neuer Stitcher, kein neuer TPT-Layer.
- keine SemanticCrystal- oder FinalizedEmission-Erzeugung.
- keine Direct Fabric -> Tensor Mutation erlauben.
- keinen MacroControlState direkt aus Resonanz oder ephemeren Fabric ableiten.

OUTPUT:

- Code, Tests, README/CHANGELOG-Update.
- validation_runs/<commit>/nctcs_closure_bundle.json
- validation_runs/<commit>/nctcs_conformance_report.json
- validation_runs/<commit>/macro_control_state.json, falls C4 erreicht.
- final_report.md mit NCTCS-Klasse und Validierungsstatus.

14 Schlussformel

$$ValidControl(PSE) \iff C4 \wedge TraceReplay = 1 \wedge GateBoundMaterialization = 1.$$

$$EmpiricalClaim(PSE) \iff ValidControl(PSE) \wedge DomainValidated = 1 \wedge EvalSchlussformel = 1.$$

Endinvariante

Der Stack darf Resonanz sehen, Kandidaten bilden, Topologien triangulieren und Tensoren aktualisieren. Kontrolle entsteht jedoch erst, wenn die gesamte Kette aus Sichtbarkeit, Kandidatur, Gate, Tensorrevision, Trace, Replay, Evaluation und realer Domaenvalidierung in einen gueltigen MacroControlState muen-det.
